

RAPPORT TECHNIQUE

Assistant conversationnel RAG

Recommandation d'événements culturels à partir des données OpenAgenda

Auteur	Aimé Endaye
Formation	Data Engineer – OpenClassrooms
Type de projet	Architecture RAG (Retrieval-Augmented Generation)
Date	Juin 2026
Version	1.0

Sommaire

1. Introduction	3
1.1. Contexte	3
1.2. Objectifs	3
1.3. Qu'est-ce que le RAG ?	3
2. Architecture générale.....	4
2.1. Composants techniques	4
2.2. Structure du projet.....	4
3. Pipeline de traitement des données	6
3.1. Collecte des données (get_openagenda_events.py).....	6
3.2. Nettoyage et préparation (preprocessing_openagenda.ipynb).....	6
3.3. Génération des embeddings (vectorisation.py).....	7
3.4. Construction de l'index vectoriel (create_faiss_index.py)	7
3.5. Validation de la recherche sémantique (test_faiss.py).....	7
4. L'assistant conversationnel RAG (chatbot_rag.py).....	9
4.1. Recherche du contexte	9
4.2. Construction du prompt.....	9
4.3. Génération de la réponse	9
4.4. Boucle d'interaction.....	9
5. Installation et configuration	10
5.1. Environnement	10
5.2. Configuration de la clé d'API	10
5.3. Exécution du pipeline complet	10
6. Résultats obtenus	11
6.1. Tests.....	11
7. Limites et perspectives	12
7.1. Limites actuelles.....	12
7.2. Pistes d'amélioration	12
8. Conclusion	13

1. Introduction

1.1. Contexte

Ce rapport documente la conception et la réalisation d'un assistant conversationnel intelligent destiné à recommander des événements culturels. Le projet s'inscrit dans le cadre de la formation Data Engineer d'OpenClassrooms et vise à mettre en œuvre une chaîne complète de traitement de données : depuis la collecte de données ouvertes jusqu'à leur exploitation par un modèle de langage.

Les données proviennent d'OpenAgenda, une plateforme collaborative qui agrège les agendas culturels d'un grand nombre d'organisations (musées, salles de spectacle, collectivités, associations). Ces données sont riches mais hétérogènes, ce qui justifie une étape de préparation soignée avant toute exploitation.

1.2. Objectifs

L'objectif principal est de permettre à un utilisateur de poser une question en langage naturel (par exemple « Donne-moi des événements pour enfants ») et d'obtenir des recommandations pertinentes d'événements réels. Les objectifs opérationnels sont les suivants :

- Collecter automatiquement les événements culturels via l'API OpenAgenda.
- Nettoyer et structurer les données pour ne conserver que les événements exploitables et récents.
- Transformer les événements en représentations vectorielles (embeddings) afin de permettre une recherche sémantique.
- Indexer ces vecteurs dans une base FAISS pour une recherche rapide par similarité.
- Mettre en place une architecture RAG qui combine la recherche sémantique et un modèle de langage pour générer des réponses fiables et contextualisées.

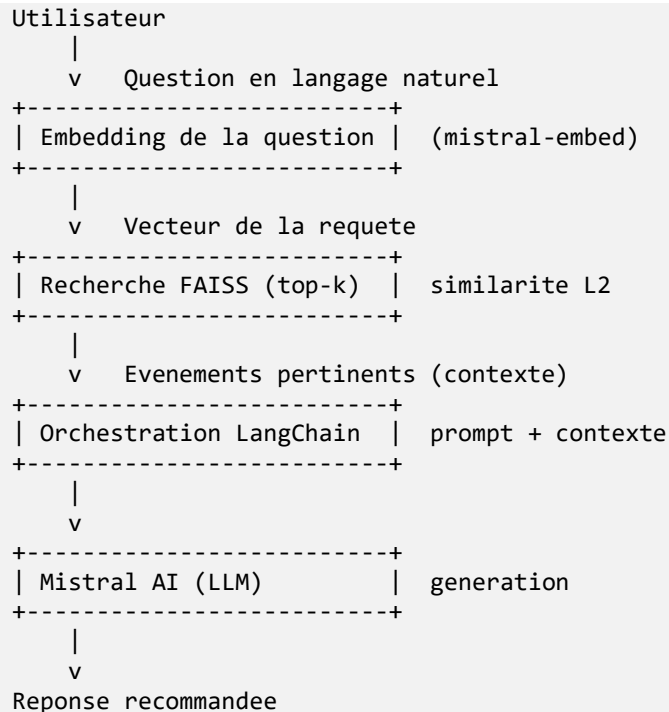
1.3. Qu'est-ce que le RAG ?

Le RAG (Retrieval-Augmented Generation, ou génération augmentée par la recherche) est une architecture qui combine deux composants complémentaires. Le premier, le module de recherche (retrieval), identifie dans une base documentaire les éléments les plus pertinents par rapport à la question posée. Le second, le modèle de langage (generation), rédige une réponse en s'appuyant exclusivement sur ces éléments.

Cette approche présente un avantage décisif : elle ancre les réponses du modèle dans des données réelles et vérifiables, ce qui réduit fortement le risque d'hallucination (génération d'informations inexacts). Dans le cadre de ce projet, le modèle ne répond qu'à partir des événements effectivement présents dans la base, garantissant des recommandations fondées sur des données authentiques.

2. Architecture générale

L'application repose sur un enchaînement de traitements organisé en pipeline. Chaque étape produit un artefact (fichier de données ou index) consommé par l'étape suivante. Le schéma ci-dessous décrit le flux complet, de la question de l'utilisateur jusqu'à la réponse générée.



2.1. Composants techniques

Chaque brique technologique remplit un rôle précis dans la chaîne :

Composant	Rôle dans le projet
OpenAgenda API	Source des données : collecte des agendas et de leurs événements culturels.
Pandas / NumPy	Manipulation, nettoyage et structuration des données tabulaires.
Mistral AI – mistral-embed	Génération des embeddings (vecteurs numériques) des événements et des questions.
FAISS	Indexation et recherche par similarité des vecteurs (recherche sémantique).
LangChain	Orchestration du pipeline RAG : assemblage prompt + contexte + modèle.
Mistral AI – mistral-small	Modèle de langage qui rédige la réponse finale à partir du contexte.
Pickle	Sérialisation des métadonnées et des embeddings sur disque.
python-dotenv	Gestion sécurisée de la clé d'API via un fichier .env.

2.2. Structure du projet

Le projet est organisé selon une arborescence claire qui sépare les données, les scripts, les notebooks et les tests :

```
rag-evenements-culturels/
|-- app/
|   |-- app.py
|-- data/
|   |-- raw/          evenements_openagenda_france.csv
|   |-- processed/   evenements_openagenda_clean.csv
|   |               evenements_openagenda_embeddings.pkl
|   |-- faiss_index/ index.faiss + metadata.pkl
|-- notebooks/       preprocessing_openagenda.ipynb
|-- scripts/         get_openagenda_events.py
|                   vectorisation.py
|                   create_faiss_index.py
|                   test_faiss.py
|                   chatbot_rag.py
|-- tests/           test_dates.py
|-- .env
|-- README.md
`-- requirements.txt
```

3. Pipeline de traitement des données

3.1. Collecte des données (get_openagenda_events.py)

La première étape consiste à interroger l'API OpenAgenda. Le script procède en deux temps : il récupère d'abord la liste des agendas disponibles (jusqu'à 1 000 agendas, par pages de 100), puis parcourt chacun de ces agendas pour collecter l'intégralité de leurs événements grâce à une pagination par offset.

Pour chaque événement, le script extrait un ensemble de champs pertinents et les normalise dans une structure tabulaire :

- Identifiants : agenda_uid, agenda, event_uid.
- Contenu : titre et description (en français).
- Temporalité : date de début, date de fin et premier créneau (firstTiming).
- Localisation : lieu, ville, région, département, pays, latitude et longitude.
- Catégorisation : mots-clés associés à l'événement.

L'ensemble est consolidé dans un DataFrame Pandas puis exporté au format CSV. Cette extraction produit un volume très important d'événements bruts (plus de 200 000 enregistrements), qui constituent la matière première du projet.

Note de sécurité : la clé d'API OpenAgenda est actuellement écrite en dur dans le script. Pour une mise en production, il est recommandé de la déplacer dans le fichier .env, au même titre que la clé Mistral, afin de ne jamais exposer de secret dans le code source.

3.2. Nettoyage et préparation (preprocessing_openagenda.ipynb)

Le notebook de prétraitement transforme les données brutes en un jeu de données propre et exploitable. Les opérations réalisées sont les suivantes :

1. Suppression des colonnes inutiles (date_debut, date_fin, region, departement, pays) afin de ne conserver que l'information utile.
2. Suppression des lignes incomplètes : tout événement sans titre, description, ville ou coordonnées géographiques est écarté.
3. Élimination des doublons sur la base de l'identifiant unique event_uid.
4. Extraction et normalisation de la date réelle de l'événement à partir du champ structuré firstTiming, puis conversion en format datetime (UTC).
5. Filtrage temporel : seuls les événements de moins d'un an (à partir de la date d'exécution) sont conservés, afin de ne recommander que des événements pertinents.

À l'issue de ce traitement, le jeu de données nettoyé conserve environ 10 000 événements, structurés en 11 colonnes, et sauvegardé dans evenements_openagenda_clean.csv. Les colonnes finales sont :

Colonne	Description
agenda_uid / agenda	Identifiant et nom de l'agenda source.

Colonne	Description
event_uid	Identifiant unique de l'événement (clé de dédoublement).
titre	Titre de l'événement.
description	Description textuelle de l'événement.
date	Date de l'événement (datetime UTC).
lieu / ville	Nom du lieu et ville d'accueil.
latitude / longitude	Coordonnées géographiques.
type_evenement	Mots-clés / catégories associés.

3.3. Génération des embeddings (vectorisation.py)

Pour permettre une recherche sémantique, chaque événement doit être transformé en vecteur numérique. Le script concatène le titre et la description de chaque événement, puis envoie ces textes au modèle mistral-embed de Mistral AI.

Afin de respecter les limites de l'API et d'optimiser les appels, le traitement est réalisé par lots (batches) de 32 textes. Une gestion robuste des erreurs est mise en place : en cas de dépassement de quota (erreur HTTP 429), le script attend 60 secondes avant de relancer la requête, ce qui garantit que l'intégralité des événements est traitée sans interruption. Une temporisation de 0,5 seconde entre chaque lot limite par ailleurs le risque de saturation.

Les embeddings obtenus sont ajoutés au DataFrame puis sauvegardés avec les métadonnées associées au format Pickle (evenements_openagenda_embeddings.pkl), prêts à être indexés.

3.4. Construction de l'index vectoriel (create_faiss_index.py)

Les embeddings sont chargés puis convertis en une matrice NumPy de type float32, format attendu par FAISS. Un index de type IndexFlatL2 est créé : il mesure la proximité entre vecteurs au moyen de la distance euclidienne (L2). Plus la distance est faible, plus l'événement est sémantiquement proche de la requête.

Tous les vecteurs sont ajoutés à l'index, qui est ensuite sauvegardé (index.faiss). En parallèle, les métadonnées de chaque événement (toutes les colonnes hors embedding) sont enregistrées dans metadata.pkl. Cette séparation permet, lors d'une recherche, de retrouver instantanément les informations lisibles d'un événement à partir de son indice dans l'index.

Choix de l'index : IndexFlatL2 effectue une recherche exhaustive (exacte) sur l'ensemble des vecteurs. Ce choix est pertinent pour un volume de l'ordre de 10 000 événements car il garantit des résultats optimaux. Pour des volumes nettement plus importants, un index approché (IVF, HNSW) offrirait un meilleur compromis vitesse/mémoire.

3.5. Validation de la recherche sémantique (test_faiss.py)

Ce script vérifie le bon fonctionnement de la recherche indépendamment du modèle de langage. Une question d'exemple (« concert gratuit à Paris ») est vectorisée via mistral-embed, puis l'index FAISS retourne les cinq événements les plus proches. Le titre, la date, la ville et un extrait de

description de chaque résultat sont affichés, ce qui permet de contrôler visuellement la pertinence de la recherche avant d'intégrer la couche de génération.

4. L'assistant conversationnel RAG (chatbot_rag.py)

Le cœur de l'application réunit la recherche sémantique et la génération de réponses. Le fonctionnement se décompose en quatre temps.

4.1. Recherche du contexte

Lorsqu'un utilisateur pose une question, celle-ci est d'abord transformée en vecteur par le même modèle mistral-embed. La fonction de recherche interroge l'index FAISS pour récupérer les k événements les plus pertinents (par défaut k = 5). Pour chaque résultat, le titre, la date, le lieu, la ville et la description sont formatés en un bloc de texte lisible, puis assemblés pour constituer le contexte transmis au modèle.

4.2. Construction du prompt

Un gabarit de prompt (ChatPromptTemplate) encadre strictement le comportement du modèle. Les consignes sont explicites : l'assistant doit répondre uniquement à partir des événements fournis dans le contexte, et signaler clairement lorsque les informations disponibles sont insuffisantes. Cette contrainte est essentielle pour prévenir les hallucinations et garantir la fiabilité des recommandations.

4.3. Génération de la réponse

LangChain assemble les composants au moyen d'une chaîne (prompt | llm | output parser). Le modèle mistral-small-latest est configuré avec une température de 0,2, valeur basse qui privilégie des réponses précises et factuelles plutôt que créatives. La réponse est diffusée en streaming, ce qui améliore l'expérience utilisateur en affichant le texte au fur et à mesure de sa génération.

4.4. Boucle d'interaction

L'assistant fonctionne en mode interactif dans le terminal : il invite l'utilisateur à saisir une question, affiche la réponse, puis répète l'opération jusqu'à ce que l'utilisateur saisisse une commande de sortie (exit, quit ou q).

Exemple d'utilisation :

Votre question : Donne-moi des evenements pour enfants

Reponse IA :

Voici quelques evenements adaptes aux enfants,
avec leur date, leur lieu et une courte description...

5. Installation et configuration

5.1. Environnement

Le projet s'appuie sur Python 3 et un environnement virtuel dédié. La mise en place s'effectue en quelques commandes :

```
python3 -m venv .venv
source .venv/bin/activate
./venv/bin/python -m pip install -r requirements.txt
```

5.2. Configuration de la clé d'API

La clé d'API Mistral est lue depuis un fichier .env placé à la racine du projet, garantissant qu'aucun secret n'apparaît dans le code :

```
MISTRAL_API_KEY=votre_cle_api
```

5.3. Exécution du pipeline complet

Les scripts s'exécutent dans l'ordre suivant :

Étape	Commande	Résultat produit
1. Collecte	python scripts/get_openagenda_events.py	data/raw/...france.csv
2. Nettoyage	notebook preprocessing_openagenda.ipynb	...clean.csv
3. Embeddings	python scripts/vectorisation.py	...embeddings.pkl
4. Indexation	python scripts/create_faiss_index.py	index.faiss + metadata.pkl
5. Test	python scripts/test_faiss.py	Top 5 événements
6. Chatbot	python scripts/chatbot_rag.py	Assistant interactif

6. Résultats obtenus

Le pipeline a été mené à son terme et produit un assistant fonctionnel. Les principaux résultats sont résumés ci-dessous :

Indicateur	Valeur
Événements collectés (bruts)	Plus de 200 000
Événements conservés après nettoyage	Environ 10 000
Modèle d'embedding	mistral-embed
Modèle de génération	mistral-small-latest
Type d'index vectoriel	FAISS IndexFlatL2
Résultats retournés par requête	5 événements les plus pertinents

L'assistant est capable de comprendre une question formulée en langage naturel, de retrouver les événements sémantiquement proches et de rédiger une recommandation structurée (date, lieu, description), strictement fondée sur les données indexées.

6.1. Tests

Un script de test (`test_dates.py`) valide la qualité du traitement des dates : il contrôle la présence de la colonne date, convertit les valeurs en datetime, et vérifie l'absence de valeurs nulles ainsi que la répartition des événements par année. Ce contrôle garantit la cohérence temporelle du jeu de données avant son exploitation.

7. Limites et perspectives

7.1. Limites actuelles

- La clé d'API OpenAgenda est codée en dur dans le script de collecte ; elle devrait être externalisée dans le fichier .env.
- L'assistant fonctionne uniquement en ligne de commande ; il n'offre pas encore d'interface graphique pour l'utilisateur final.
- L'index FAISS est exhaustif (IndexFlatL2), ce qui reste performant à cette échelle mais deviendrait coûteux pour des millions d'événements.
- La recherche s'appuie sur la similarité sémantique seule ; elle n'intègre pas de filtrage explicite par date, ville ou gratuité.

7.2. Pistes d'amélioration

- Développer une interface web (le fichier app/app.py est prévu à cet effet, par exemple avec Streamlit ou FastAPI).
- Ajouter un filtrage hybride combinant la recherche sémantique et des filtres structurés (date, localisation, catégorie).
- Mettre en place une actualisation automatique et périodique des données OpenAgenda.
- Évaluer la qualité des recommandations à l'aide de métriques dédiées et de retours utilisateurs.
- Externaliser tous les secrets et conteneuriser l'application (Docker) pour faciliter le déploiement.

8. Conclusion

Ce projet démontre la mise en œuvre complète d'une chaîne de traitement de données moderne, depuis la collecte de données ouvertes jusqu'à leur exploitation par un modèle de langage. L'architecture RAG retenue permet de combiner la richesse des données OpenAgenda avec la capacité de compréhension du langage naturel, tout en maîtrisant le risque d'hallucination grâce à un ancrage strict dans les données réelles.

Sur le plan des compétences de Data Engineer, le projet couvre l'ensemble du cycle de vie de la donnée : ingérer (API OpenAgenda), préparer (nettoyage et filtrage), transformer (embeddings), stocker et indexer (FAISS), puis servir (assistant conversationnel). Les perspectives d'amélioration identifiées – interface web, filtrage hybride, actualisation automatique et industrialisation – ouvrent la voie à une version pleinement opérationnelle.

Aimé Endaye — Projet RAG événements culturels — Formation Data Engineer, OpenClassrooms